AI Platform — AWS Setup Guide

What to deploy, and what to send back. Roughly 45 minutes, most of it waiting.

What you are being asked to do

Deploy a small, serverless document-question-answering system into an AWS account, then return one configuration file so the requester can use it. They have no console access, so every value they need has to come back in that file.

What gets created	What does *not*
A Bedrock Knowledge Base, an S3 bucket for documents, an S3 bucket for conversation state, a Lambda function, a Bedrock Guardrail, and IAM roles scoped to those resources.	No VPC. No load balancer. No database. No cluster. Nothing that runs when idle — Lambda bills per invocation and S3 Vectors bills per query.

> **Cost while idle is close to zero.** Storage only, in the order of cents per month. Cost appears when questions are asked, billed as Bedrock tokens. A budget alert before the first deploy is recommended.

You will need

- An AWS account, and permission to manage Bedrock, S3, Lambda and IAM in it.
- A region where Bedrock is available — `us-west-2` and `us-east-1` are the usual choices.
- About 45 minutes, most of which is the deploy running unattended.

STEP 1 Enable Bedrock model access

> **Do this first. It is the step that fails silently.** If model access is not enabled, everything below still succeeds — the deploy completes, no error appears — and then the very first question fails with `AccessDeniedException`. It looks like a broken application and is not one.

In the AWS console, in the region you have chosen:

1. Open **Amazon Bedrock** → **Model access**.
2. Choose **Modify model access**.
3. Enable a Claude chat model. If several are available, enable more than one — it makes changing the choice later a config edit rather than another ticket.
4. Enable **Titan Text Embeddings V2** (`amazon.titan-embed-text-v2`). Nothing becomes searchable without it.
5. Submit, and wait until status reads *Access granted*.

Confirm from the command line:

```
aws bedrock list-inference-profiles --region us-west-2 \
  --query "inferenceProfileSummaries[?contains(inferenceProfileId,'anthropic')].inferencePr

aws bedrock list-foundation-models --region us-west-2 \
  --query "modelSummaries[?contains(modelId,'titan-embed-text-v2')].modelId"
```

Both commands must return at least one entry. If either is empty, stop here.

STEP 2 Install the tools

Tool	Why	Install
Python 3.11+	Runs the deploy scripts	`python.org`
uv	Package manager used by the Makefile	`curl -LsSf https://astral.sh/uv/install.sh | sh`
Node 22+	Runs the AWS CDK CLI	`nodejs.org`
AWS CLI v2	Credentials and reading outputs	`aws.amazon.com/cli`
Docker	Packages the Lambda	`docker.com` — must be *running*

Then get the code and install it:

```
git clone <repository-url>
cd AWS-GenAI-Platform
make install
```

STEP 3 Point the AWS CLI at the account

```
aws configure sso        # if the account uses SSO
aws configure            # otherwise

aws sts get-caller-identity     # confirm: prints the account id
export AWS_REGION=us-west-2     # same region as Step 1
```

STEP 4 Deploy

A tenant is one isolated set of resources — its own knowledge base, its own function, its own IAM role. The repository ships with an example tenant called `acme`; the requester will tell you which slug they want. Substitute it for `acme` throughout.

```
cd infra && npx cdk bootstrap     # once per account and region
cd .. && make deploy
```

This takes 10–20 minutes, mostly building the Knowledge Base. It is safe to leave unattended. It is also safe to re-run: CDK applies only differences.

> **If it fails on permissions** — the most likely failure — the error names the missing action. Grant it and re-run `make deploy`; nothing needs undoing first.

STEP 5 Generate the configuration file

There is a command for this. It reads every value back out of the deployed stacks and writes them to a file called `.env`, so nothing has to be copied by hand:

```
make env TENANT=acme
cat .env
```

You should see roughly this, with real values:

```
TENANT=acme
AWS_REGION=us-west-2
KNOWLEDGE_BASE_ID=ABCD1234EF
DOCUMENTS_BUCKET=aiplat-knowledge-acme-documents-...
SESSION_BUCKET=aiplat-api-acme-sessions-...
GUARDRAIL_ID=abcd1234efgh
GUARDRAIL_VERSION=1
AGENT_FUNCTION_URL=https://....lambda-url.us-west-2.on.aws/
```

> **If a line is missing or blank**, that stack did not deploy. Do not send a partial file — go back to Step 4 and check for errors. A missing `GUARDRAIL_ID` in particular means the system would run with no PII masking and no grounding check.

STEP 6 Add credentials

The generated file describes *what* to talk to. The recipient also needs permission to talk to it. Choose one:

Option A — SSO or an assumable role (preferred)

Nothing secret travels. Grant the recipient access, send them the SSO start URL separately, and add nothing to the file. They will run `aws configure sso` themselves.

Option B — an access key

Create a principal that can call Bedrock, read the documents bucket, and read/write the session bucket. **It does not need permission to deploy anything.** Append two lines to `.env`:

```
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
```

STEP 7 Check it works before sending

Load a few documents and ask a question about them:

```
make image-ingest

docker run --rm -v ~/.aws:/root/.aws:ro -v "$PWD/sample-docs:/data:ro" \
  -e AWS_REGION -e DOCUMENTS_BUCKET -e KNOWLEDGE_BASE_ID \
  aiplat-ingest /data --wait

make ask Q="What do these documents cover?"
```

A grounded answer with `[1]`-style citations means the whole chain works. An honest *"I don't know"* for something the documents do not cover is also a correct result — the system is built to refuse rather than guess, and seeing it do so is a good sign.

STEP 8 Send the file back

> **If you chose Option B, this file now contains live credentials.** Send it through whatever your organisation uses for secrets — a password manager, a vault, an encrypted share. Not email, not chat.

The recipient saves it as `.env` in the repository root. That is the entire handover: they have no further setup to do.

If something goes wrong

Symptom	Cause	Fix
`AccessDeniedException` when asking a question	Model access not enabled, or enabled in a different region	Step 1, in the region from Step 3
`make deploy` fails naming an IAM action	The deploying principal lacks that permission	Grant it, re-run — nothing needs undoing

Docker or bundling error during deploy	Docker is not running	Start Docker, re-run
`make env` writes an empty or partial file	Some stacks did not deploy	Re-run `make deploy` and read the errors
Questions return "I don't know" for everything	No documents indexed yet	Run the ingest step; wait for it to finish
Deploy succeeds, questions fail with a throttling error	Bedrock quota too low for this account	Request a quota increase for the chosen model

---

Removing it again

```
make destroy
```

The documents bucket and the knowledge base are deliberately retained, so an accidental teardown cannot delete an indexed corpus. Delete those two by hand when the removal is intended.

Full technical detail, including the exact IAM permissions and the answers a security review will ask for, is in `docs/aws-requirements.md` in the repository.